Thesis Plan

Performance, Safety, and Engineering Trade-offs of Rust and C++ in Nokia RAN Control-Plane Systems

Ha Vien
University of Helsinki, Master's Programme in Computer Science (Software study track)
Nokia instructors: Keijo Heljanko, Christoph Luneburg, Petteri Kotimaa

9 August 2026

1 Background and motivation

Modern RAN control-plane software (the L3 of a gNB, e.g. the gNB-CU-CP [1]) is written mostly in C++, which delivers the throughput and latency such systems need but carries well-known memory-safety and concurrency risks (use-after-free, data races, double-free). These classes dominate real defect populations, and migrating to memory-safe languages is now an explicit policy position [11, 3]. Rust offers comparable performance with safety guaranteed at compile time on machine-checked foundations [9].

Its fit for latency-critical telecom control planes is unproven. The "Rust performs like C++" consensus rests on general-purpose benchmarks [13] that report average-case behaviour; closer studies find real costs, both in networking code [5] and in specific mechanisms such as bounds and overflow checking [18]. For a RAN control plane the operative metric is not the average but the tail [4]. This thesis is carried out at Nokia on a subsystem modelled on a real Nokia gNB-CU-CP component.

2 Research gap

Two master's theses at Ericsson are the closest prior work. Karlbäck and Orö compare Rust and Go against C++ for RAN applications and conclude C++ remains the choice where performance is critical [10]. Arvidsson and Kaller measure median and 99th-percentile latency for 5G control-plane procedures across five asynchronous frameworks, and find that the Rust runtimes match C++ at the median but their p99 rises more steeply past a threshold rate [2].

Neither isolates the language: in both, the runtime and concurrency model vary together with it, so a measured difference cannot be attributed to the language rather than to the framework built on it. The gap is therefore a comparison that holds the specification, algorithms, concurrency model, allocator, and build settings fixed, *and* sweeps past the saturation knee, *and* measures what incremental migration costs.

3 Objective and research questions

The thesis builds two functionally equivalent implementations of a UE Context Management service in modern C++ and in Rust and compares them under a spec-justified synthetic signalling workload. Since similar average throughput is already known, it targets four less-explored dimensions:

RQ1 How do Rust and C++ compare in latency predictability (mean, p99, p99.9, jitter) under increasing load and overload, with the concurrency model held fixed?

RQ2 Which classes of C++ memory and concurrency defects does safe Rust eliminate at compile time, at what stage are the rest detected, and which classes does it *not* prevent?

RQ3 What overhead, if any, does Rust's safety model impose on throughput, CPU, and memory?

RQ4 What are the practical implications (effort, FFI interoperability, risk) of adopting Rust incrementally in an existing C++ RAN codebase?

RQ2 is deliberately two-sided: a catalogue containing only defects Rust prevents would confirm its own premise, so it spans the boundary: five classes safe Rust rejects and two it does not. Memory-safety defects in real Rust code invariably trace back to `unsafe` [16], while deadlock remains an open problem for the language [17].

4 Methodology

A single language-agnostic specification (public API, UE state machine, data structures, sharding and concurrency model) is frozen before any implementation code is written, so the comparison measures language rather than design differences. Both implementations use identical algorithms, a shared-nothing sharded worker-thread model, the same allocator, and comparable build optimisation. Holding the concurrency model fixed matters because thread-per-core structuring itself changes tail latency [6]. Both must pass the same parity suite.

The workload is swept from nominal load to overload on pinned, frequency-locked hardware, with interleaved repeats and bootstrap confidence intervals [8], latency captured in high-dynamic-range histograms against an intended send schedule so coordinated omission does not hide the tail [15]. Evaluation adds a controlled fault-injection study [12] and an FFI prototype. On the FFI, the plan takes a narrow position: confining `unsafe` to the seam is a statement about what the compiler checks, not run-time protection, since host and component share an address space [14]. Only synthetic data is used, so no personal or confidential data is processed.

5 Expected contributions

- A reproducible, telecom-relevant benchmarking framework.
- A frozen shared specification and two parity-tested implementations.
- A quantitative comparison emphasising tail latency under overload.
- A defect-class taxonomy mapping both what the compile-time guarantee covers and where it stops.
- Evidence-based guidelines for incremental Rust adoption, where the dominant cost is the learning curve rather than run-time [7].

6 Status and timeline

Work has not started yet: no literature review, no implementation, and no measurements. The timeline below begins in June 2026 with topic definition and foundational learning, then moves to drafting in July.

Item	State
Literature review	Not started
Spec + implementations	Not started
RQ2 (fault injection)	Not started
RQ4 (FFI prototype)	Not started
RQ1 / RQ3 (benchmarks)	Not started
Thesis document	Not started

Month (2026)	Goal	What happens
June	Foundations	Define the topic; learn C++, Rust, and 5G basics; begin the literature review.
July	Draft	Draft this plan; start writing thesis chapters and outline the evaluation design.
August	Build	Freeze the shared specification; implement both versions and the parity suite; build the benchmark harness.
September	Measure	Provision a bare-metal testbed; run the full campaign; generate figures and numbers.
October	Resolve	Finalise RQ1/RQ3 analysis; confirm or retract the tail claim; revise affected chapters; draft to supervisors and Nokia instructors.
November	Submit	Final revisions, colloquium presentation, submission.

Risks. If the testbed is delayed, RQ1 and RQ3 may remain preliminary; a cloud bare-metal instance or a Nokia lab host is the fallback. If the tail difference proves unresolvable even on bare metal, it is reported as a bounded null result with the confidence interval stated, which is a legitimate finding. Single-author skill asymmetry is bounded by the frozen specification and parity suite, and acknowledged as residual for performance.

Out of scope. The full gNB, PHY/L2 and the user plane, real captured traces, managed-language alternatives, and production operational concerns.

References

[1] 3GPP. NG-RAN; architecture description. Technical Specification (TS) 38.401, 3rd Generation Partnership Project (3GPP), 2022. V17.5.0, Release 17.

[2] A. Arvidsson and M. Kaller. Evaluating asynchronous programming frameworks for 5G control plane procedures. Master's thesis, Linköping University, Department of Computer and Information Science, 2025. Carried out at Ericsson AB.

[3] Cybersecurity and Infrastructure Security Agency and National Security Agency and Federal Bureau of Investigation and others. The case for memory safe roadmaps: Why both C-suite executives and technical experts need to take memory safe coding seriously. Cybersecurity information sheet, CISA, NSA, FBI, ASD's ACSC, CCCS, NCSC-UK, NCSC-NZ, CERT NZ, Dec. 2023.

[4] J. Dean and L. A. Barroso. The tail at scale. *Communications of the ACM*, 56(2):74–80, 2013.

[5] P. Emmerich, S. Ellmann, F. Bonk, A. Egger, E. García Sánchez-Torija, T. Günzel, S. Di Luzio, A. Obada, M. Stadlmeier, S. Voit, and G. Carle. The case for writing network drivers in high-level programming languages. In *Proceedings of the 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*, 2019.

[6] P. Enberg, A. Rao, and S. Tarkoma. The impact of thread-per-core architecture on application tail latency. In *Proceedings of the 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*, pages 1–8, 2019.

[7] K. R. Fulton, A. Chan, D. Votipka, M. Hicks, and M. L. Mazurek. Benefits and drawbacks of adopting a secure programming language: Rust as a case study. In *Proceedings of the 17th Symposium on Usable Privacy and Security (SOUPS)*, pages 597–616. USENIX Association, 2021.

[8] A. Georges, D. Buytaert, and L. Eeckhout. Statistically rigorous Java performance evaluation. In *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*, pages 57–76, 2007.

[9] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer. RustBelt: Securing the foundations of the Rust programming language. *Proceedings of the ACM on Programming Languages*, 2(POPL):66:1–66:34, 2018.

[10] R. Karlbäck and A. Orö. Holistic view on alternative programming languages for radio access network applications in cloud and embedded deployments. Master's thesis, Linköping University, Department of Computer and Information Science, 2021. Carried out at Ericsson AB.

[11] M. Miller. Trends, challenges, and strategic shifts in the software vulnerability mitigation landscape. BlueHat IL, Microsoft Security Response Center (MSRC), 2019. Approximately 70% of Microsoft CVEs are memory-safety issues.

[12] R. Natella, D. Cotroneo, and H. S. Madeira. Assessing dependability with software fault injection: A survey. *ACM Computing Surveys*, 48(3):44:1–44:55, 2016.

[13] R. Pereira, M. Couto, F. Ribeiro, R. Rua, J. Cunha, J. P. Fernandes, and J. Saraiva. Energy efficiency across programming languages: How do energy, time, and memory relate? In *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering (SLE)*, pages 256–267, 2017.

[14] E. Rivera, S. Mergendahl, H. Shrobe, H. Okhravi, and N. Burow. Keeping safe Rust safe with Galeed. In *Proceedings of the 37th Annual Computer Security Applications Conference (ACSAC)*, pages 824–836, 2021.

[15] G. Tene. HdrHistogram: A high dynamic range histogram, and the coordinated omission problem. `http://hdrhistogram.org/`, 2015. Accessed 2026.

[16] H. Xu, Z. Chen, M. Sun, Y. Zhou, and M. R. Lyu. Memory-safety challenge considered solved? an in-depth study with all Rust CVEs. *ACM Transactions on Software Engineering and Methodology*, 31(1):3:1–3:25, 2021.

[17] Y. Zhang, K. Zhang, and G. Liu. Static deadlock detection for Rust programs. arXiv preprint arXiv:2401.01114, 2024.

[18] Y. Zhang, Y. Zhang, G. Portokalidis, and J. Xu. Towards understanding the runtime performance of Rust. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 140:1–140:6, 2022.